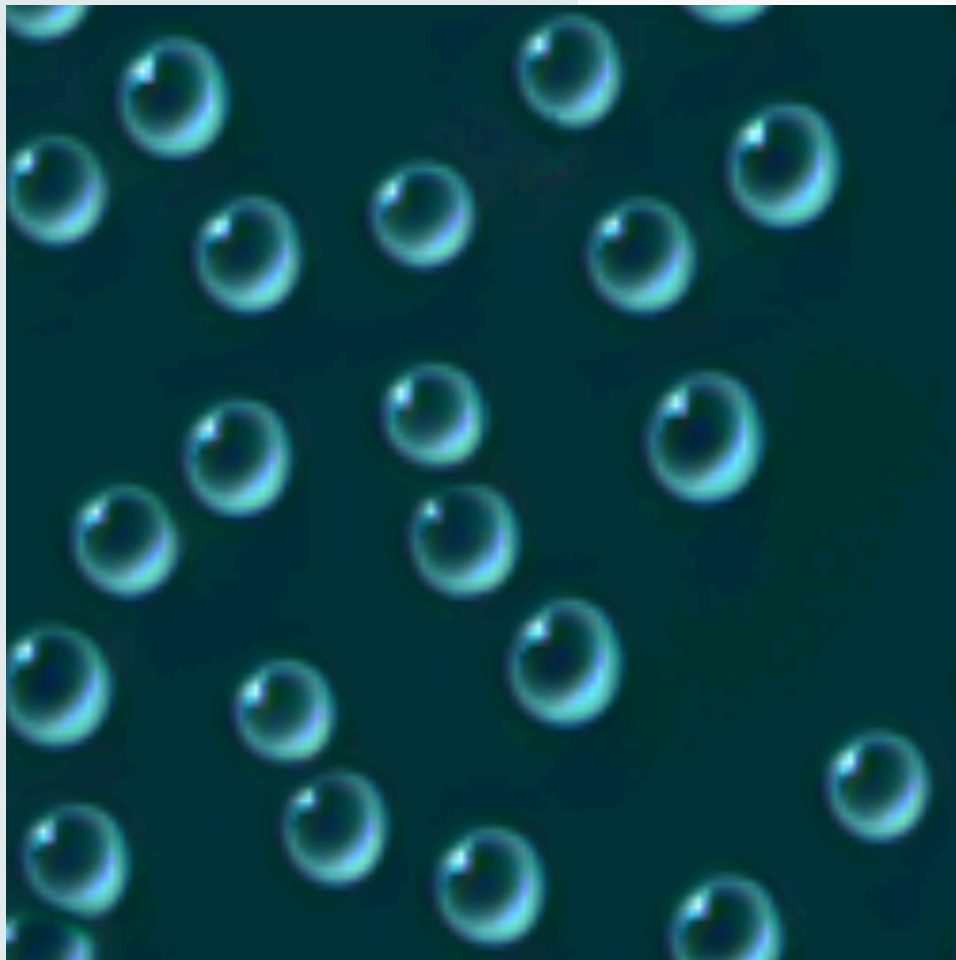


*Alexander Mordvintsev,
Eyvind Niklasson, and
Ettore Randazzo.*

Texture Generation with
Neural Cellular Automata



Projet **20**

IMO6

Synthèse de textures par automates cellulaires

Héloïse Gouarné, Baptiste Lefebvre, Sarah Daher, Sarah Langlois

Encadré par Yann Gousseau

Implémentation de la génération statique

Implémentation “**from scratch**” en s’appuyant sur l’article “Texture generation with neural cellular automata”. Premier résultat comportant des **fausses couleurs** et des **reproduction irrégulières** de la texture.

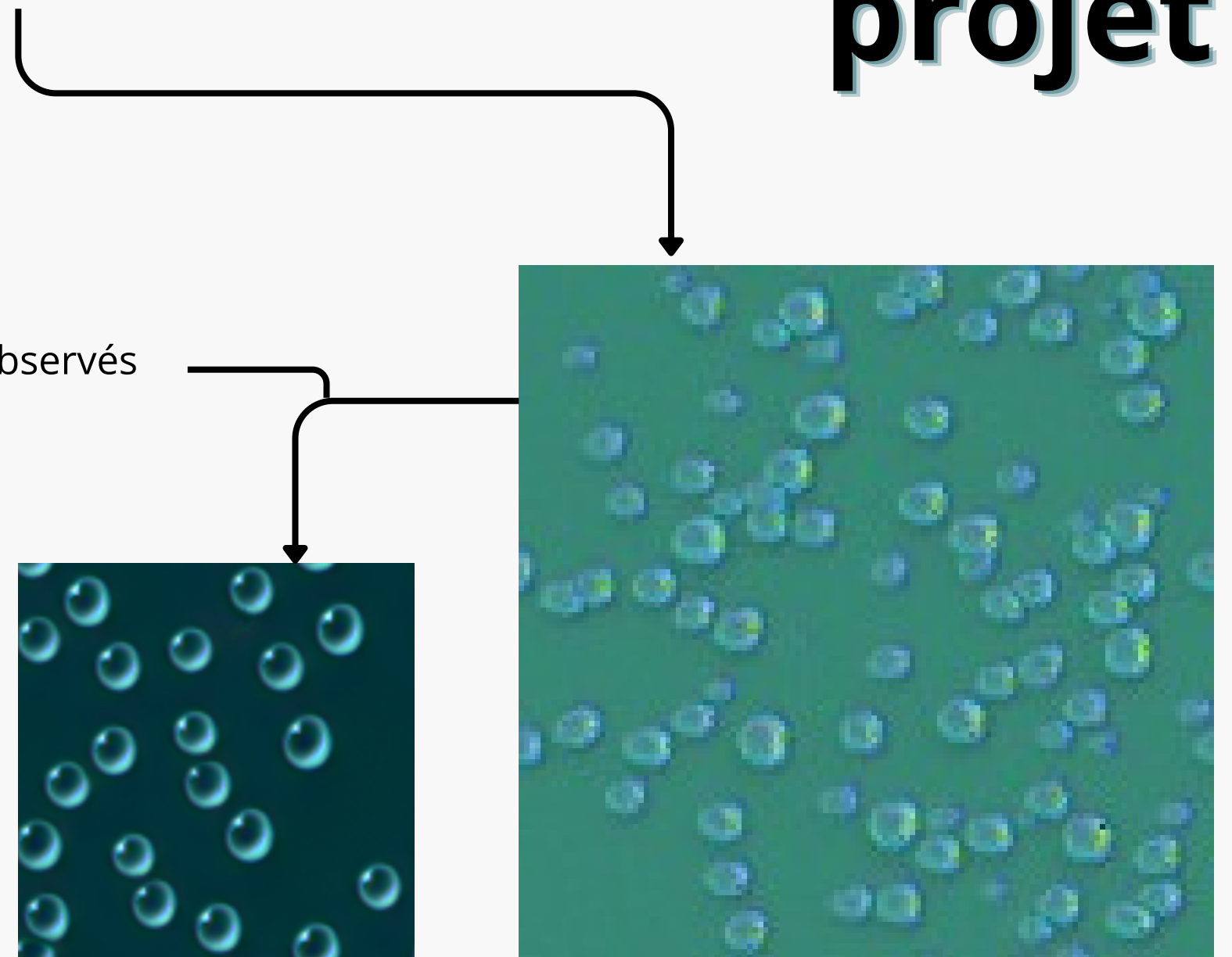
Débuggage

Phase de tests de nombreuses modifications pour solutionner les problèmes observés dans la première implémentation.

Améliorations

Ajouts du modèle de génération **multi-texture**, **Sliced-Optimal-Transport**...

Approche du projet



Modèle NCA

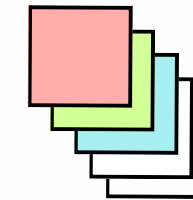
Entraînement

Simulation de l'automate cellulaire

Fonction coût VGG

Rétropropagation

Vecteur d'état d'un pixel: $\mathbf{s} \in \mathbb{R}^C$



Sur chaque image d'un *batch* de 4 images tirées aléatoirement dans la *pool* :

- calcul du vecteur de perception:

$$\mathbf{p}_t = \text{concat}(\mathbf{s}_t, K_x * \mathbf{s}_t, K_y * \mathbf{s}_t, K_{lap} * \mathbf{s}_t)$$

- mise à jour stochastique des cellules:

$$\mathbf{s}_{t+1} = \mathbf{s}_t + f(\mathbf{p}_t) \delta_{x,y,t}$$

- réinjection dans la *pool*.

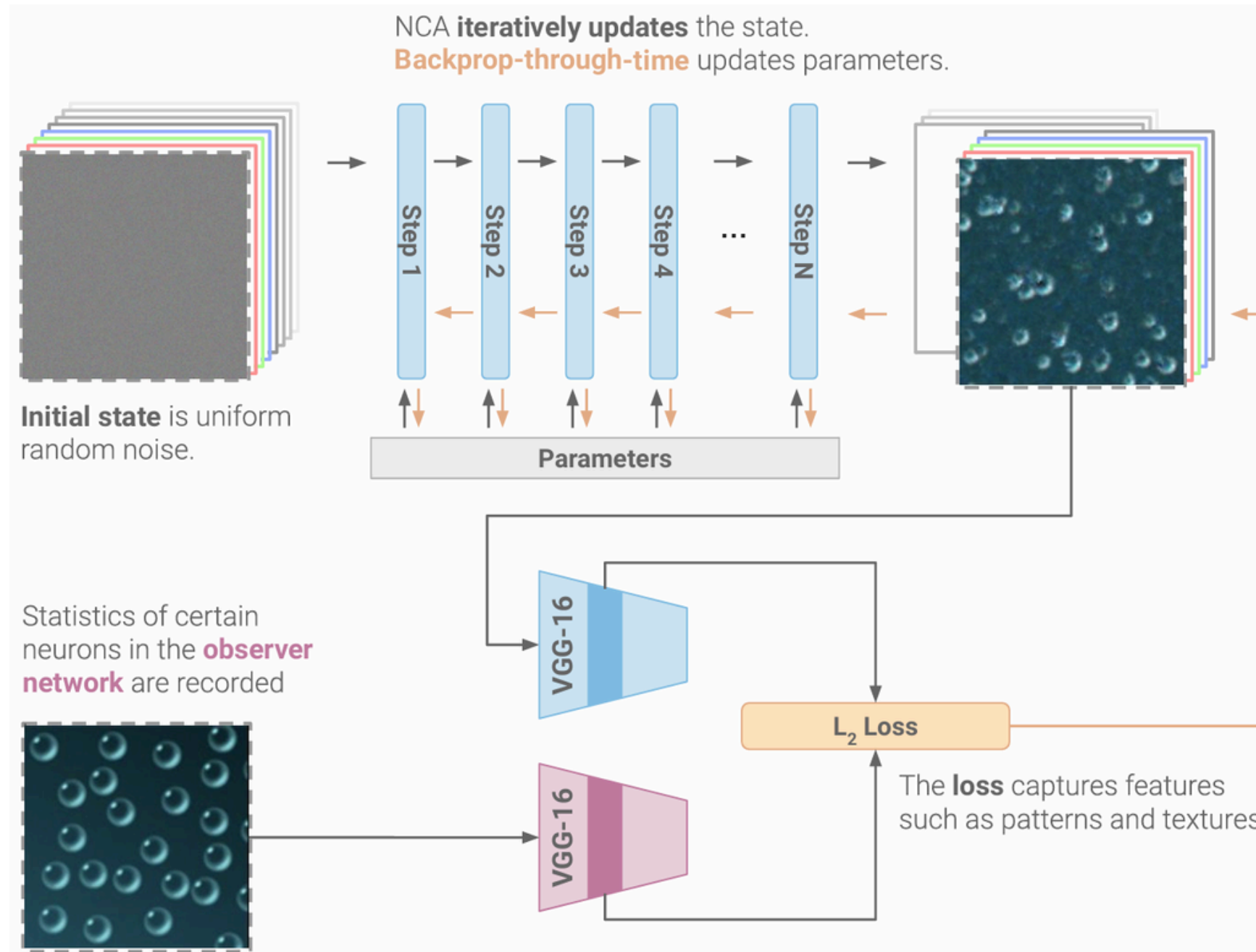


Figure 2. Texture NCA training iteration with VGG-based texture loss.

Modèle NCA

Entraînement

Simulation de l'automate cellulaire
Fonction coût VGG
Rétropropagation

$$f(\mathbf{p}) = \text{ReLU}(\mathbf{p}W_0 + b_0)W_1 + b_1$$

5868 paramètres

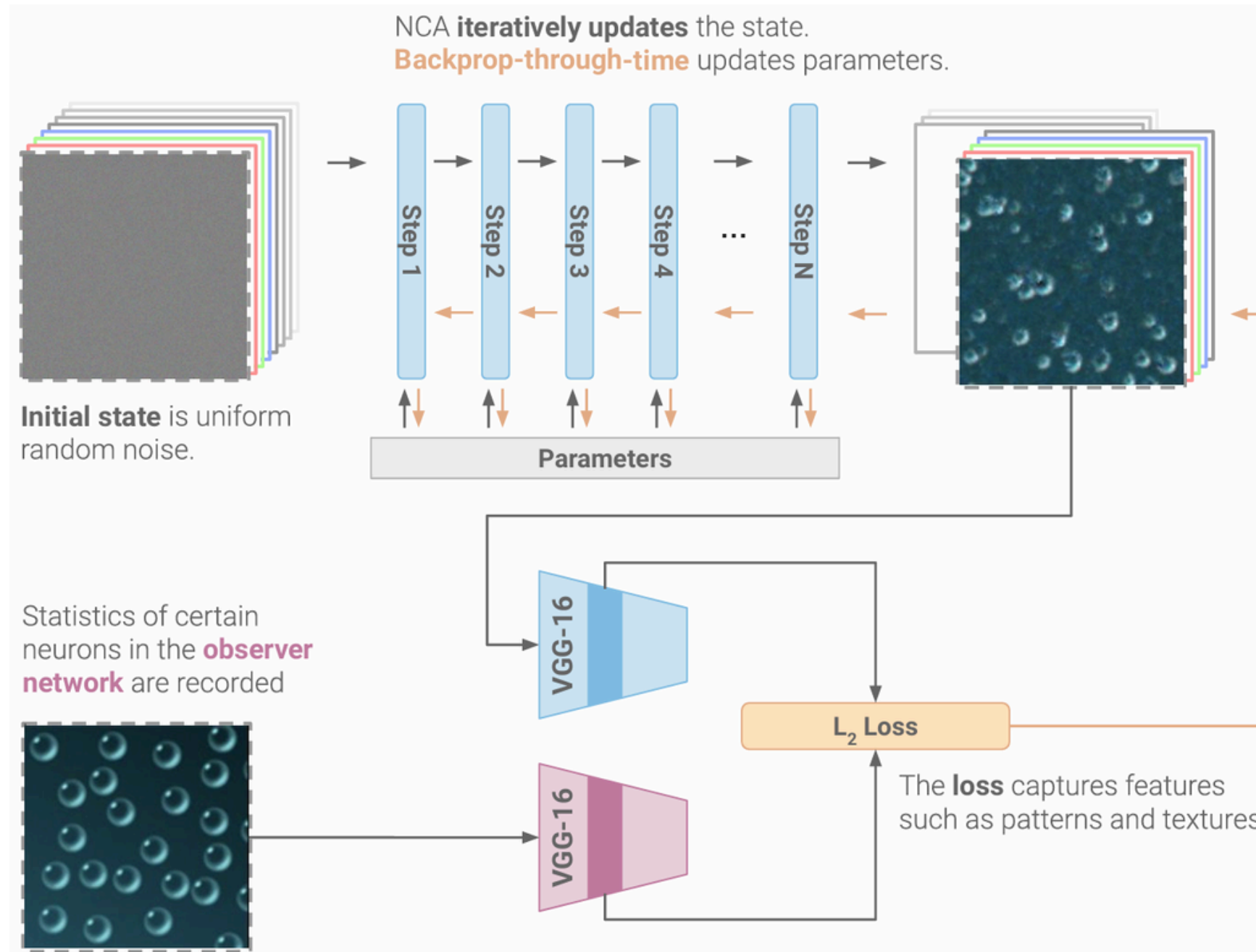


Figure 2. Texture NCA training iteration with VGG-based texture loss.

Modèle NCA

Entraînement

Simulation de l'automate cellulaire
Fonction coût VGG
 Rétropropagation

Matrice de Gram
 corrélations entre
 features

$$G = \frac{1}{HW} F F^T$$

Loss

poids proportionels au nb de
 canaux

$$L = \sum_l w_l \|G_l^{\text{pred}} - G_l^{\text{target}}\|^2$$

couches 1, 6, 11, 18

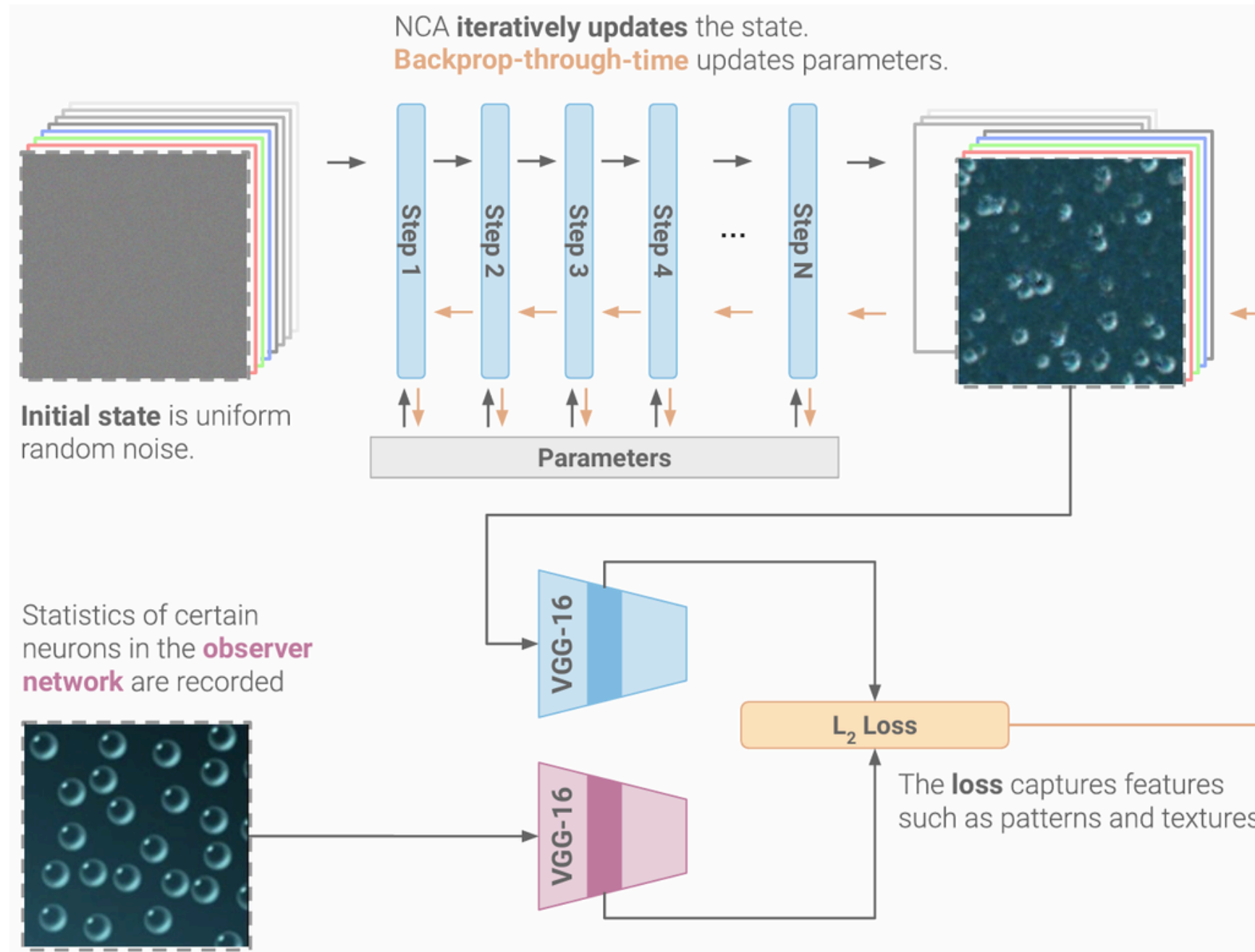


Figure 2. Texture NCA training iteration with VGG-based texture loss.

Modèle NCA

Entraînement

Simulation de l'automate cellulaire
Fonction coût VGG
Rétropropagation

Mise à jour des poids avec:

- optimiseur Adam;
- *learning rate* $\simeq 10^{-3}$ - 10^{-4}

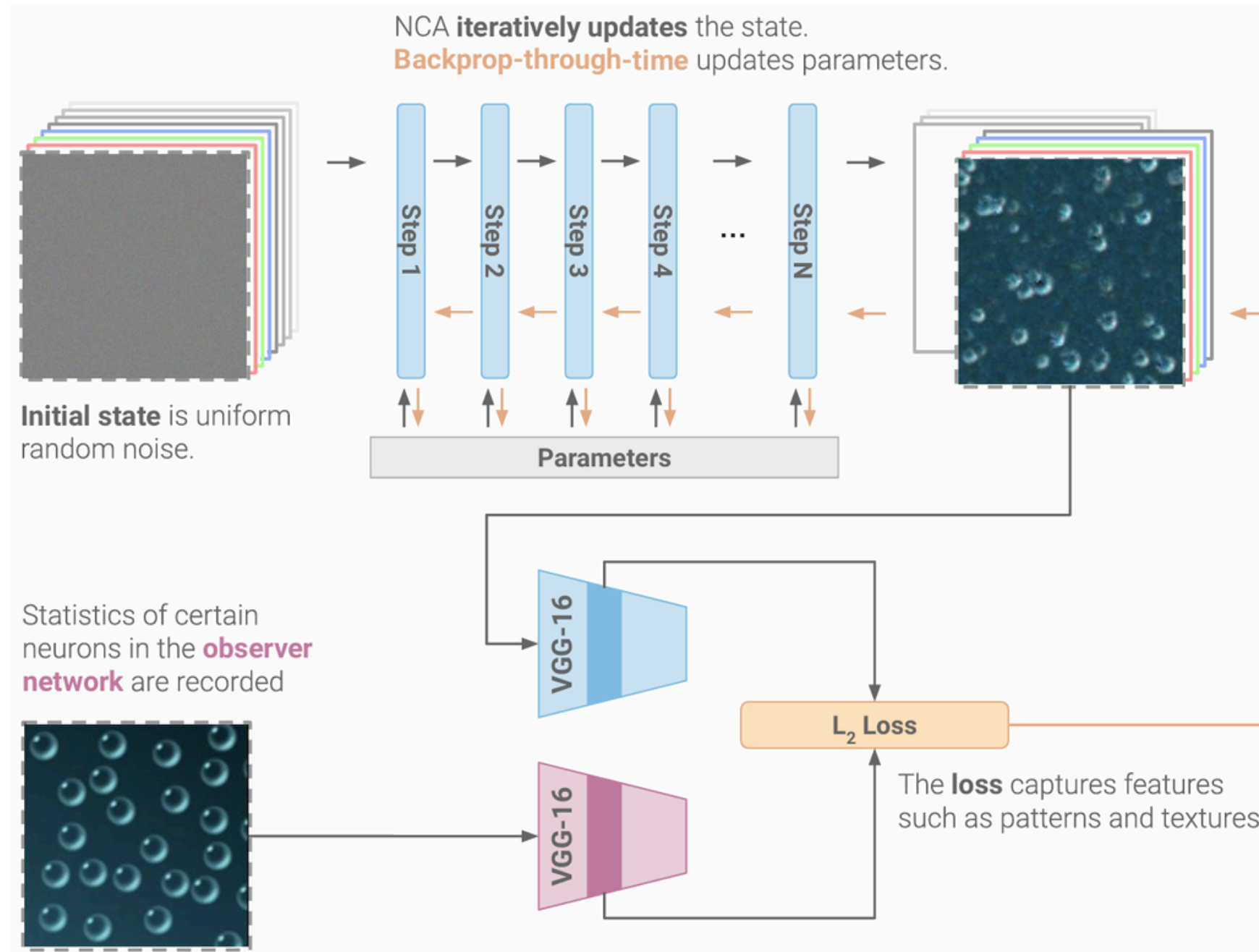
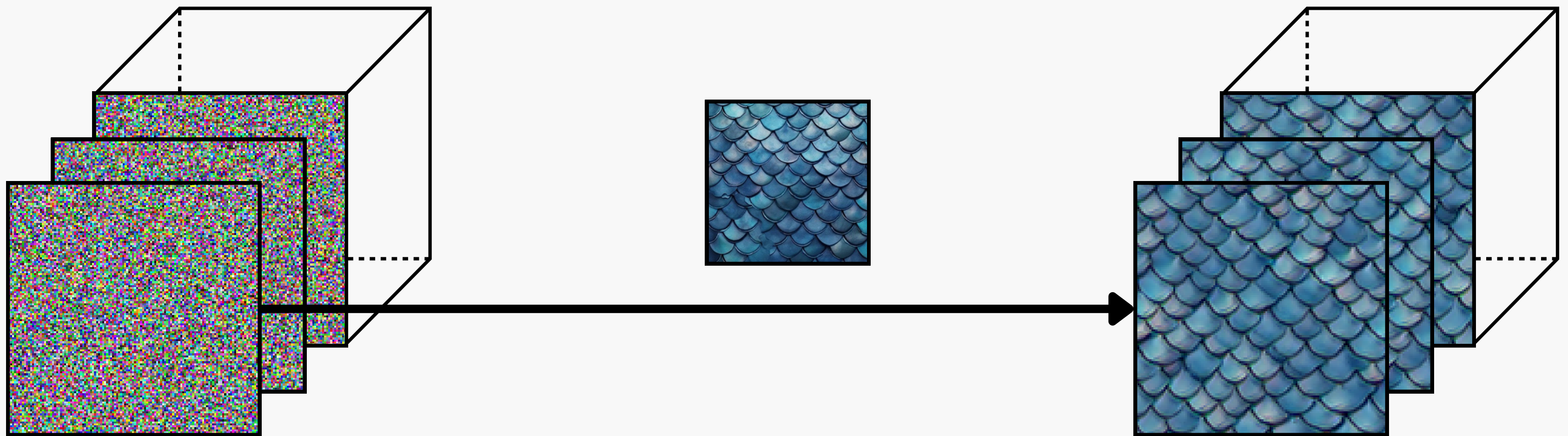


Figure 2. Texture NCA training iteration with VGG-based texture loss.

Modèle NCA

Inférence



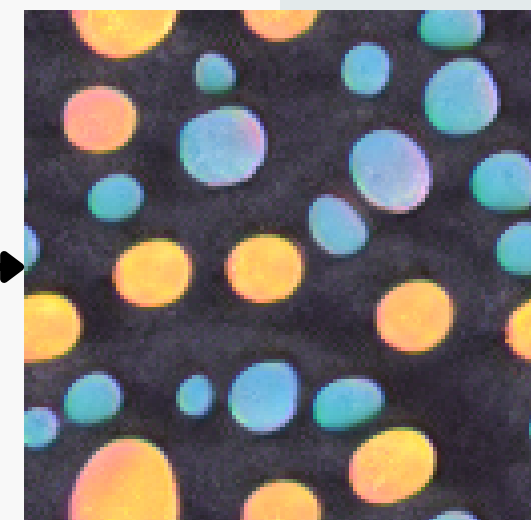
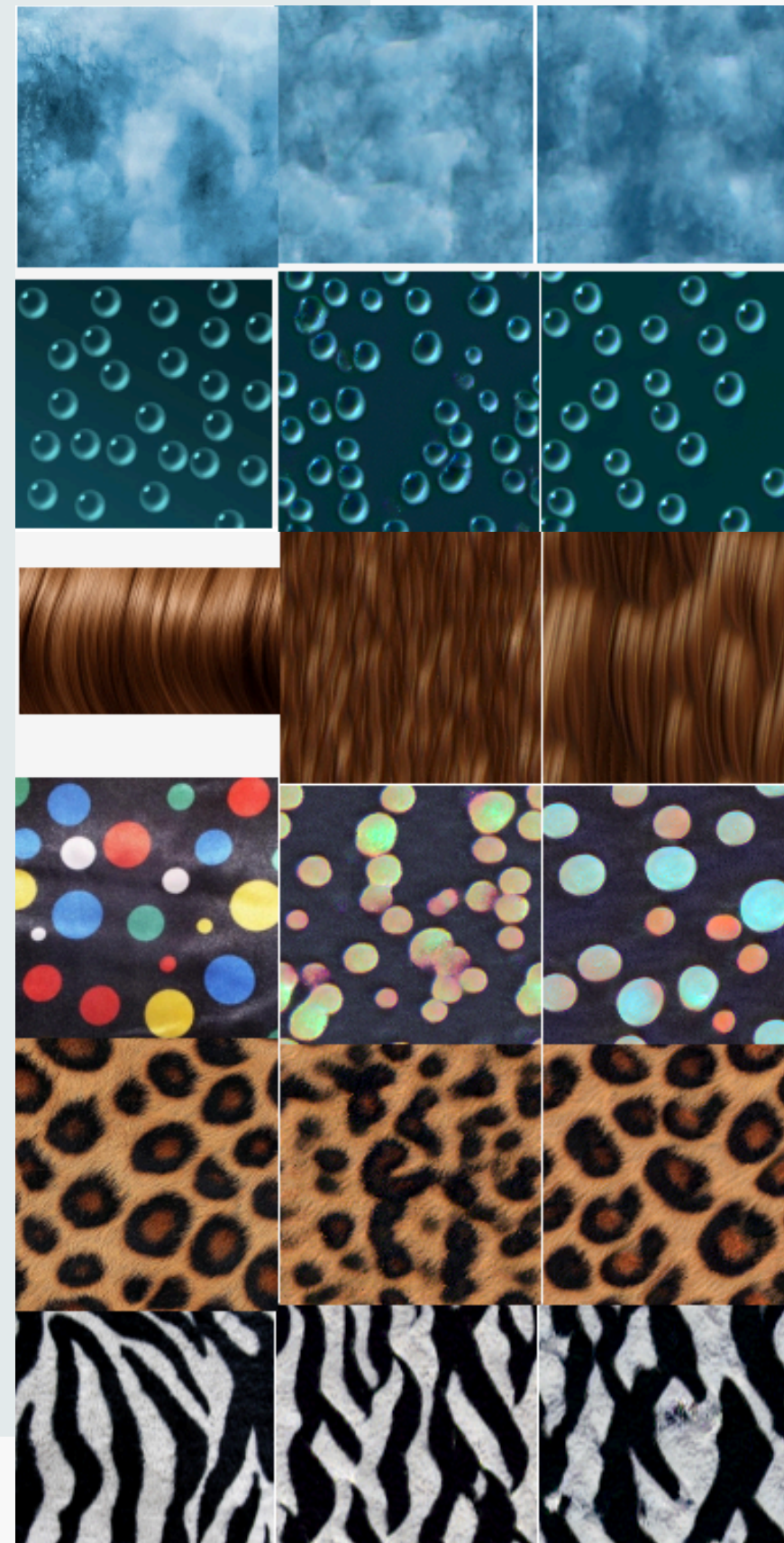
10
graines avec bruit uniforme

200
itérations du NCA par graine

10
images de texture générées

Premiers résultats

Source - 5000 steps - 10000 steps



Légère amélioration mais le modèle reste inadapté

Nécessité des 10 000 pas

Les **textures complexes**, comme la fourrure de **léopard**, nécessitent un entraînement d'au moins 10 000 pas pour bien capturer les détails et les tâches.

Difficultés sur les structures larges

Les filtres et les traitement du modèle sont **locaux**. Les textures comme celles des **cheveux**, qui nécessitent une **cohérence spatiale** sur l'ensemble de l'image, ne sont pas forcément adaptées, et vont former des version plus locales de la texture globale.

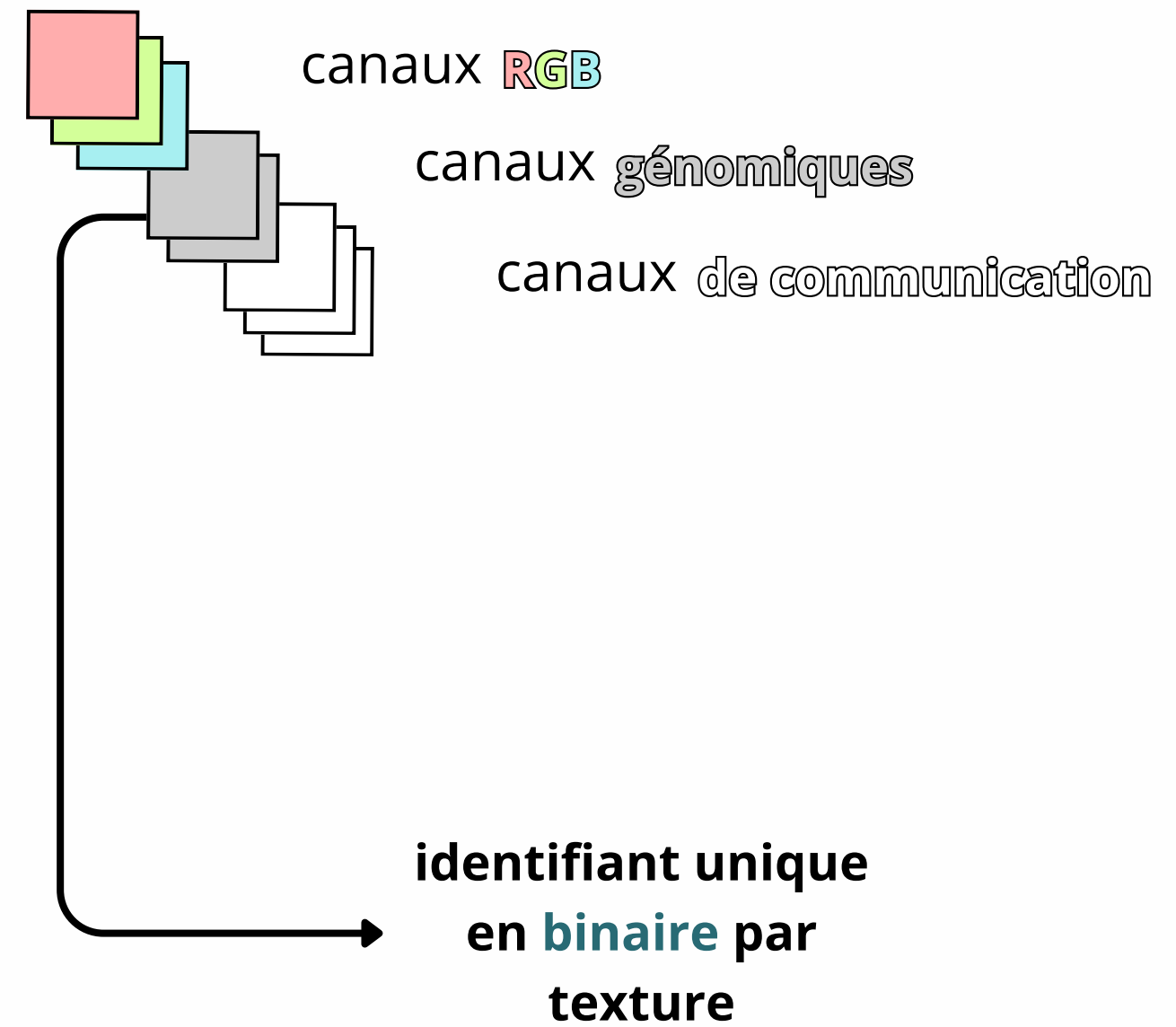
Difficultés sur la texture "dotted"

Cette texture comporte une grande **variété de couleurs et de structures**, sans grande cohérence spatiale (les ronds peuvent se trouver n'importe où, leur placement n'est pas répétitif). Le modèle génère donc des **fausses couleurs** en moyennant les tâches de la structures sources. En constatant une nette amélioration entre 5 000 et 10 000 pas, nous avons tenté de réentraîner le modèle sur **plus de pas**.

Améliorations

Synthèse multi-texture

Vecteur d'état d'un pixel: $s \in \mathbb{R}^C$



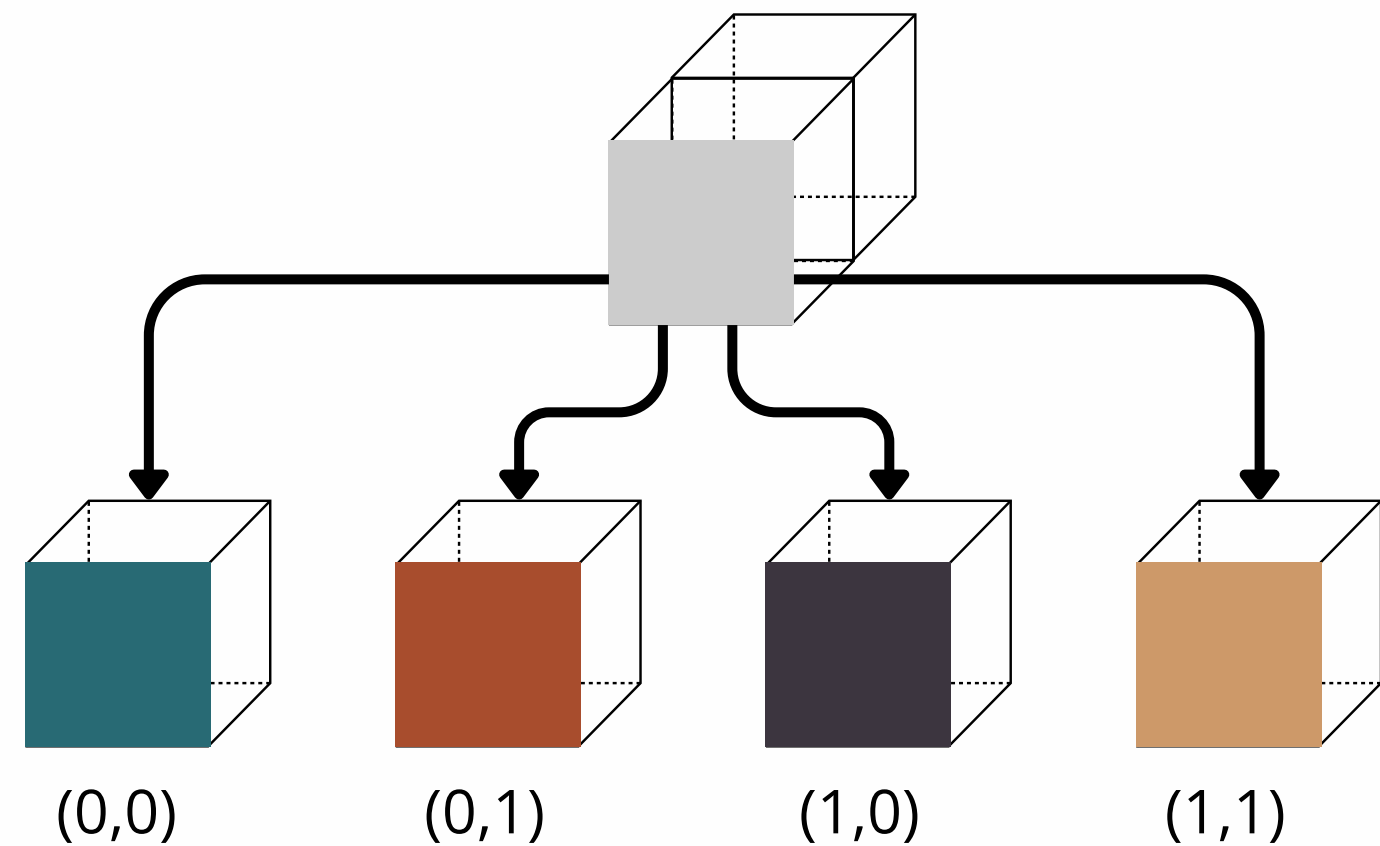
Améliorations

Synthèse multi-texture

Vecteur d'état d'un pixel: $s \in \mathbb{R}^C$



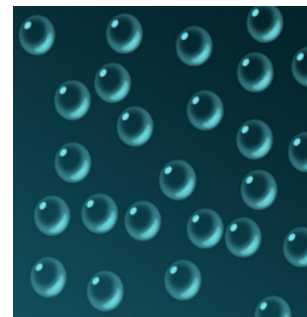
Division de la *pool* :



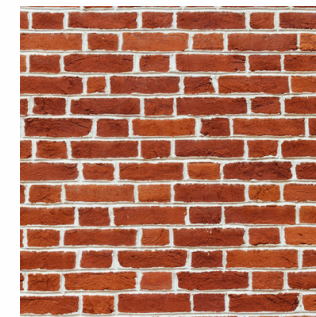
Améliorations

Synthèse multi-texture

(0,0)



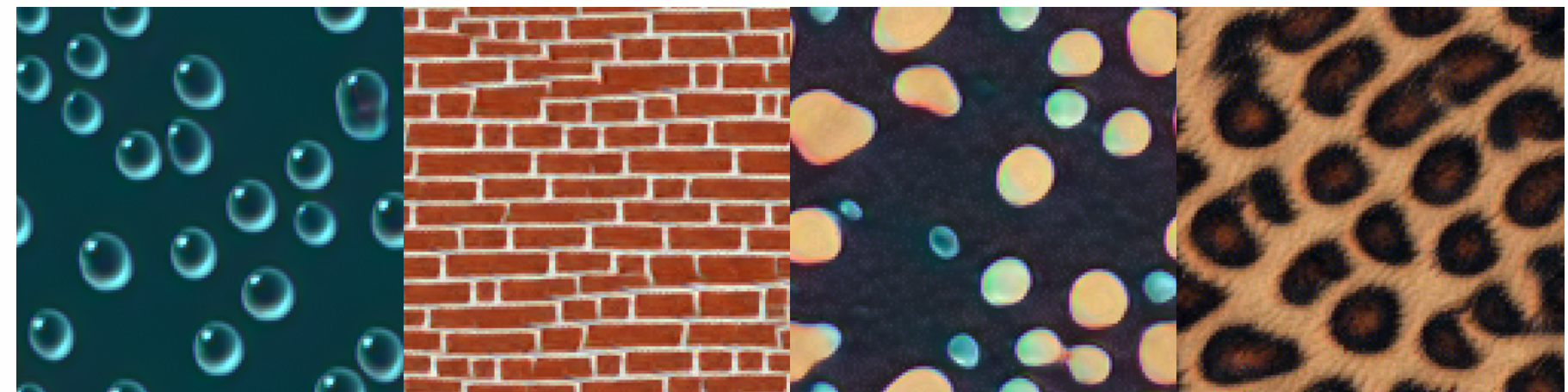
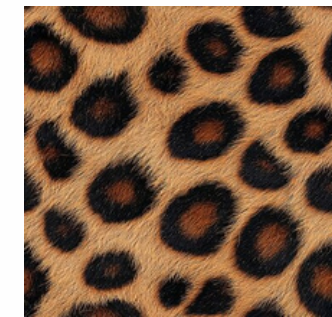
(0,1)



(1,0)



(1,1)



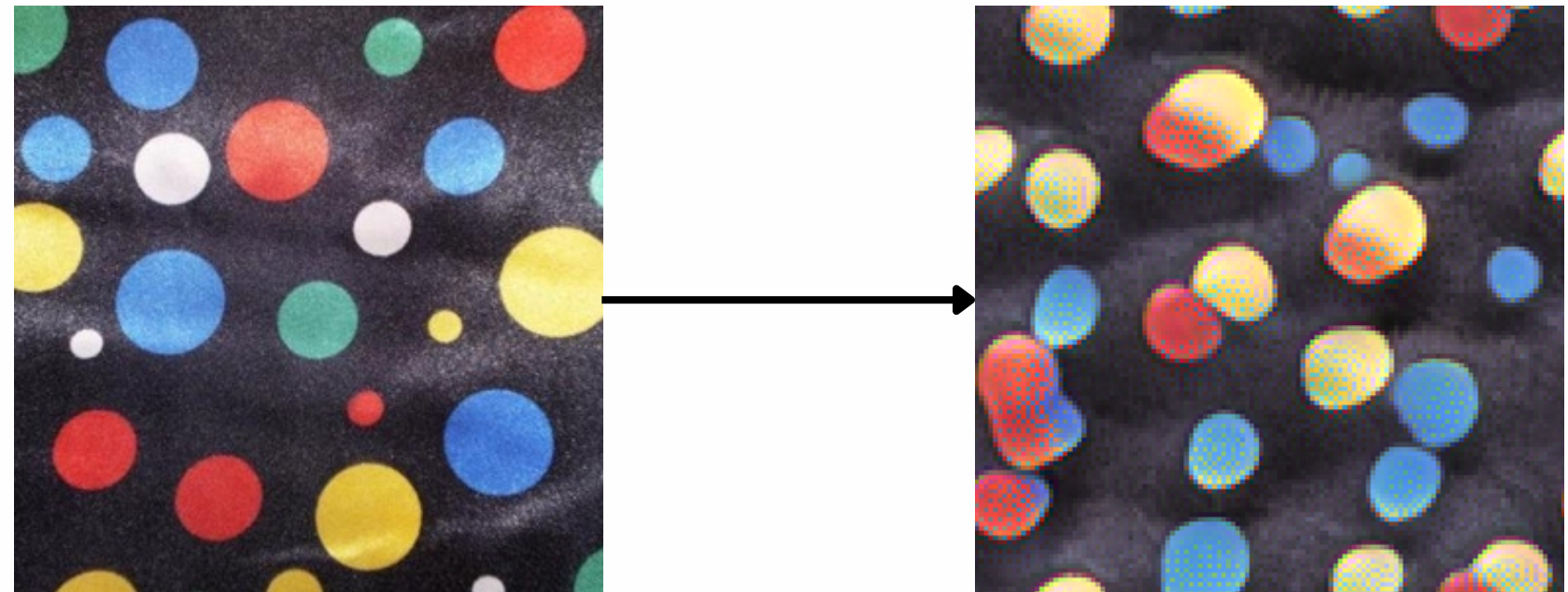
4 textures cibles, 40000 itérations d'entraînement

Améliorations

Sliced Optimal Transport

- Appliquer SOT sur les features VGG au lieu de calculer les matrices de Grams
- Ajout RGB en plus des features

$$\mathcal{L}(X, Y) = \mathbb{E}_{\theta} [W_1(\theta^{\top} X, \theta^{\top} Y)]$$



Tests et conclusions

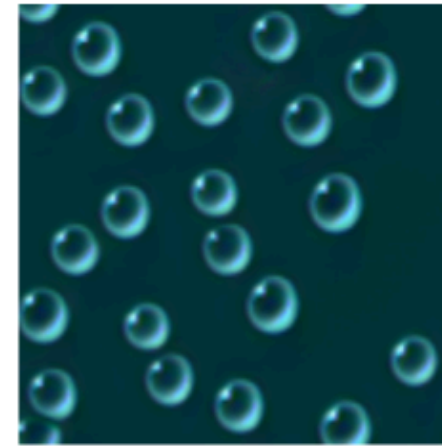
Filtres

Configurations testées :

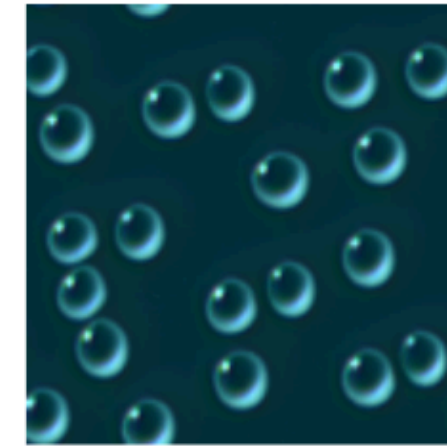
- I + **Sobel** + Laplacien
- I + **Prewitt** + Laplacien
- I + **Scharr** + Laplacien

Avec filtres aléatoires :

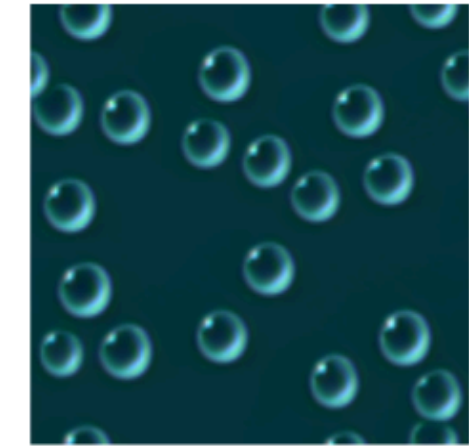
- I + filtres aléatoires de moyenne $\neq 0$
- I + filtres aléatoires de moyenne $= 0$
- Filtres aléatoires de moyenne $= 0$



(a) Sobel

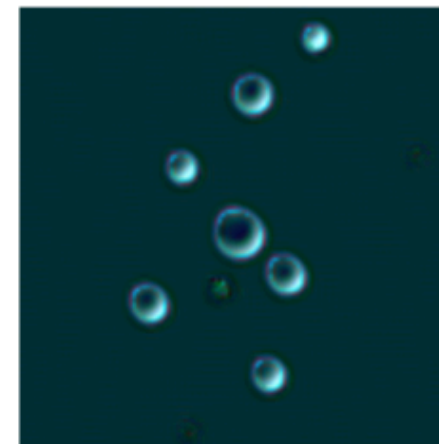


(b) Prewitt

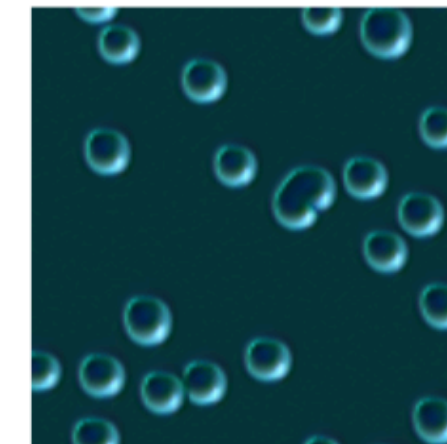


(c) Scharr

FIGURE 5 – Tests sur différentes configurations de filtres différentiels.



(a) Moyenne non nulle
avec filtre *I*



(b) Moyenne nulle
avec filtre *I*

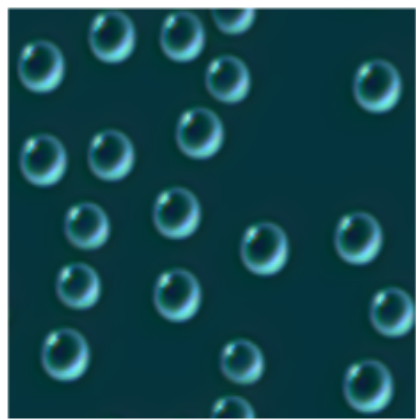


(c) Moyenne nulle
sans filtre *I*

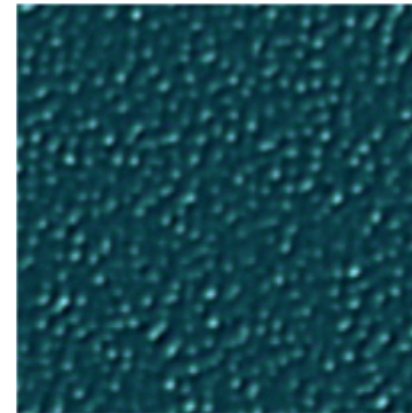
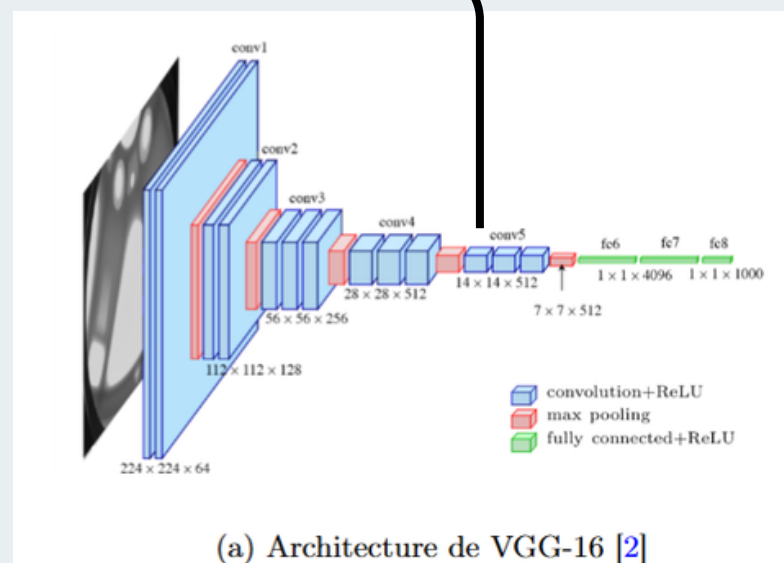
FIGURE 7 – Tests des différentes configurations de filtres aléatoires.

Tests et conclusions

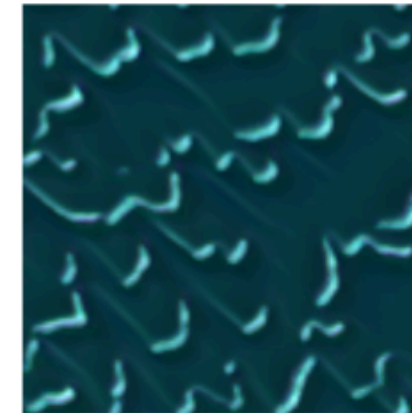
Couches VGG



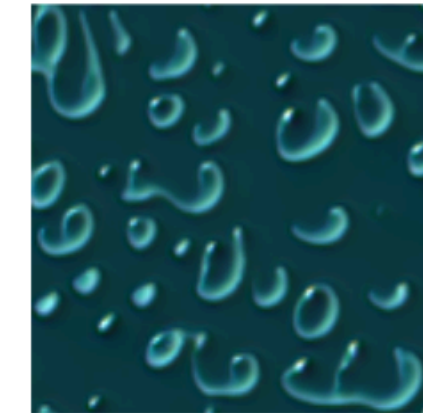
(b) Couches 1, 6, 11, 18 et 25



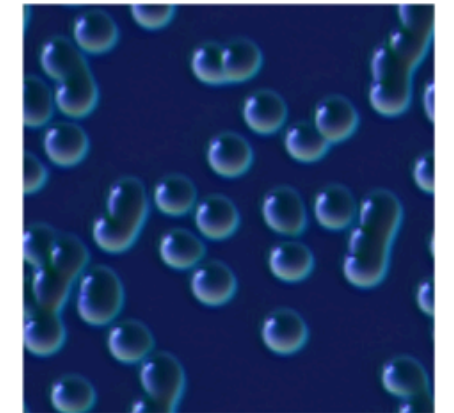
(a) Couche 1



(b) Couche 6

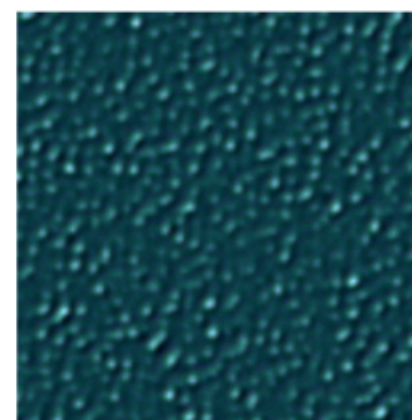


(c) Couche 11

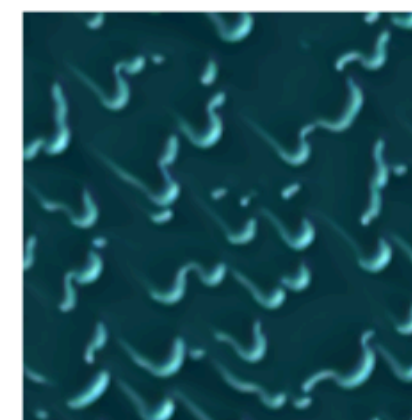


(d) Couche 18

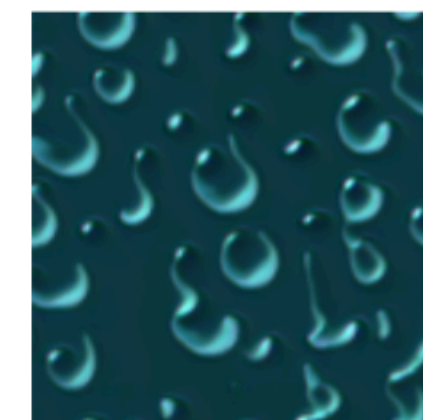
FIGURE 11 – Textures générées en ne gardant qu'une seule couche dans le calcul de la *loss*.



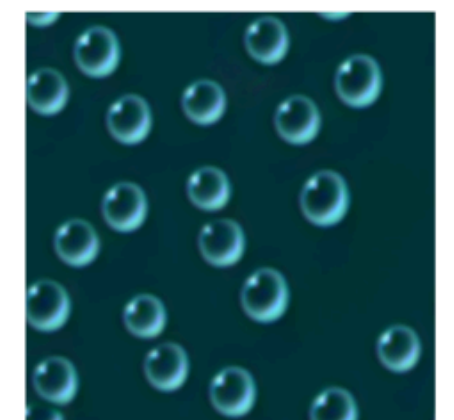
(a) Couche 1



(b) Couches 1,6



(c) Couches 1,6,11



(d) Couches 1,6,11,18

FIGURE 12 – Textures générées en accumulant graduellement les couches dans le calcul de la *loss*.

Tests et conclusions

Mémoire GPU et texture haute résolution

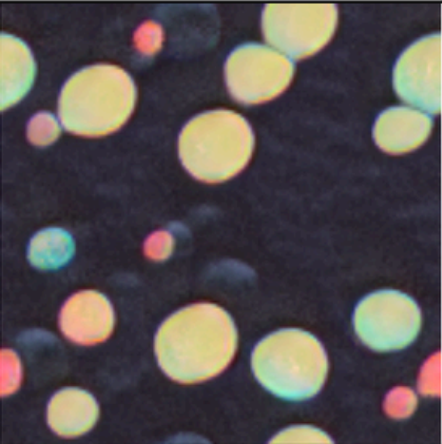
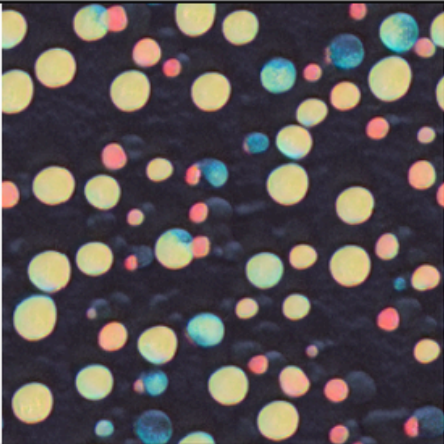
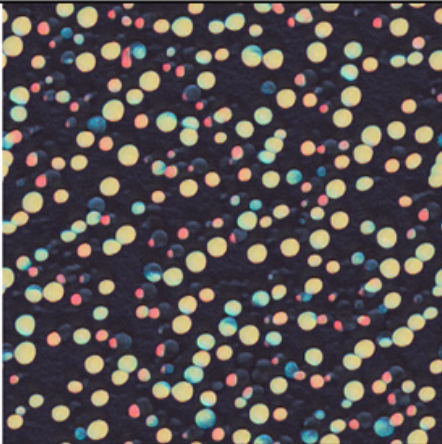
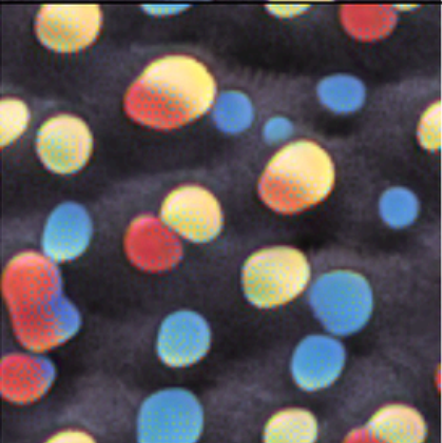
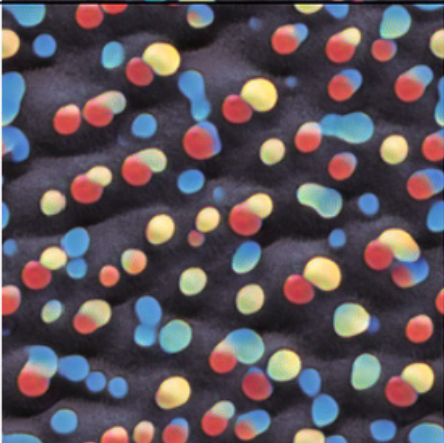
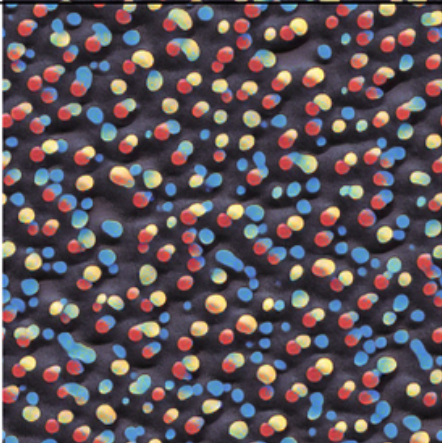
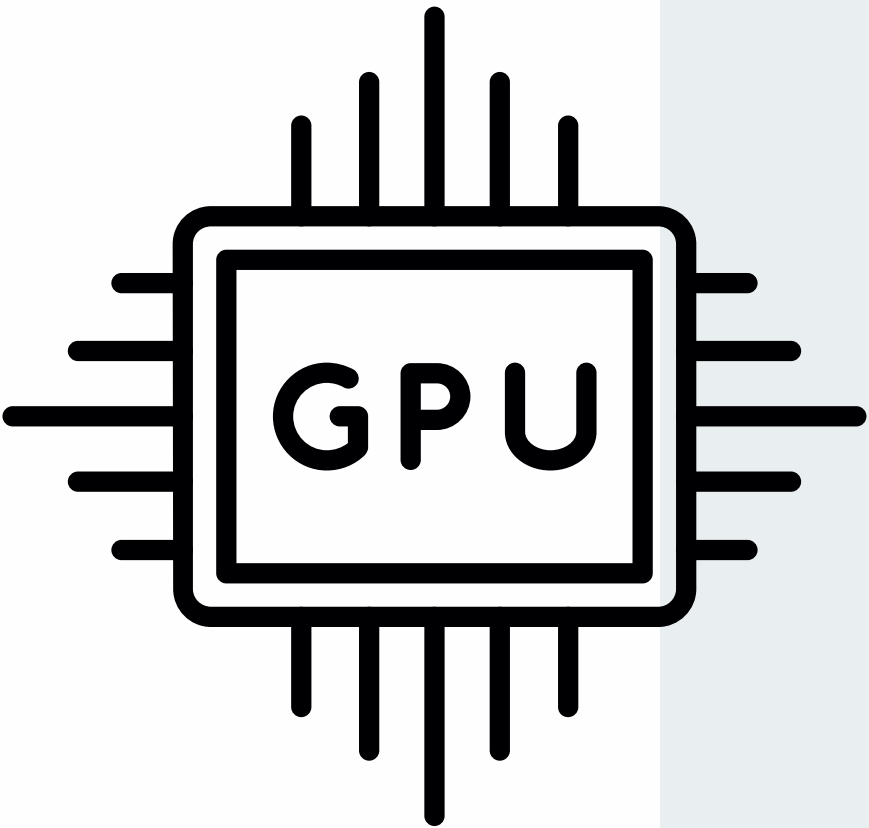
	128 × 128	256 × 256	512 × 512
Gram <i>loss</i>			
SoT <i>loss</i>			

TABLE 3 – Images générées à partir d’un modèle entraîné sur la texture *dotted* en résolution 128 × 128, en faisant varier la résolution à l’inférence.



Tests et conclusions

Reconstruction après dommage

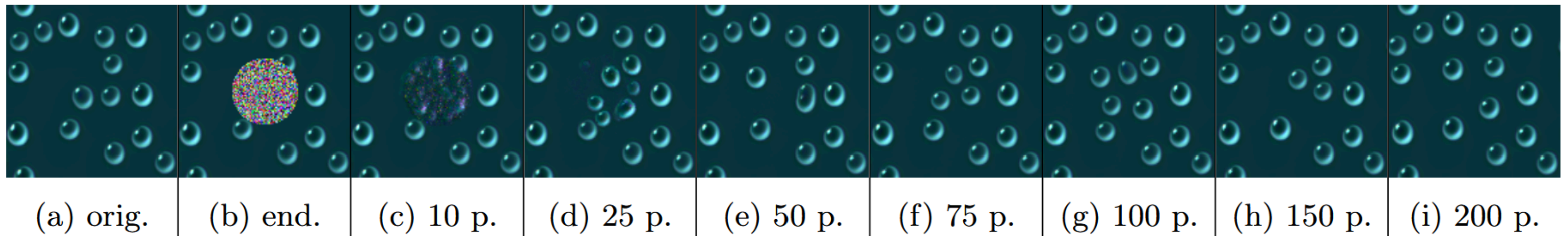
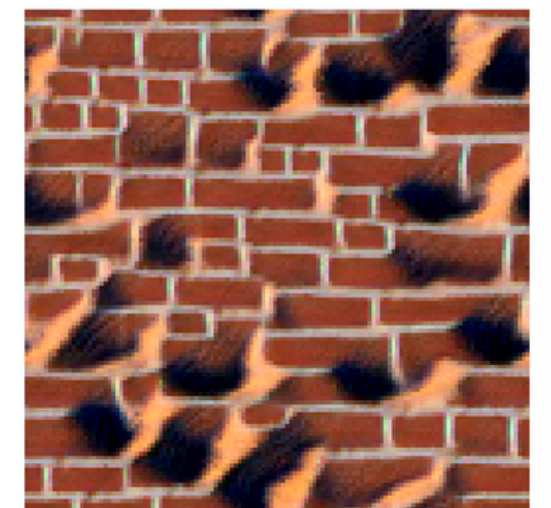
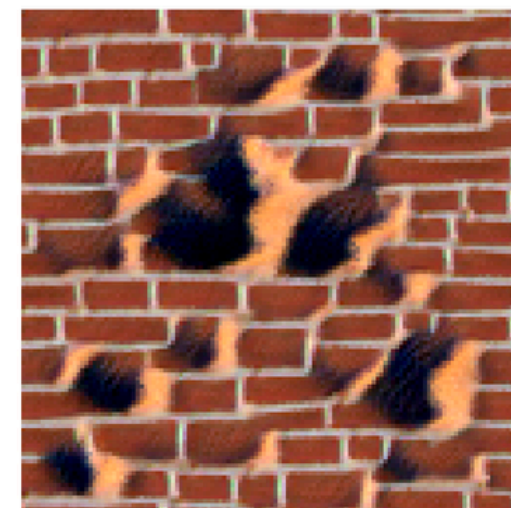
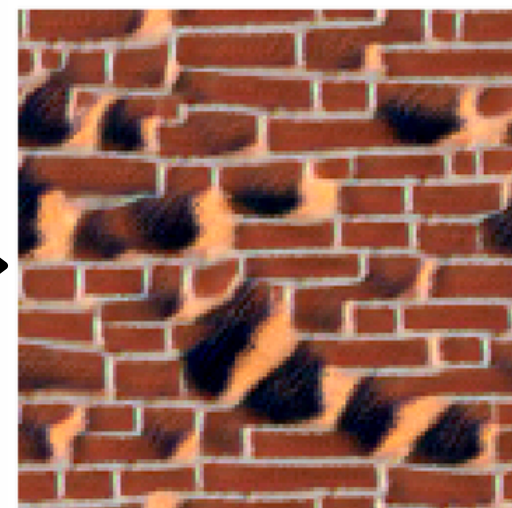
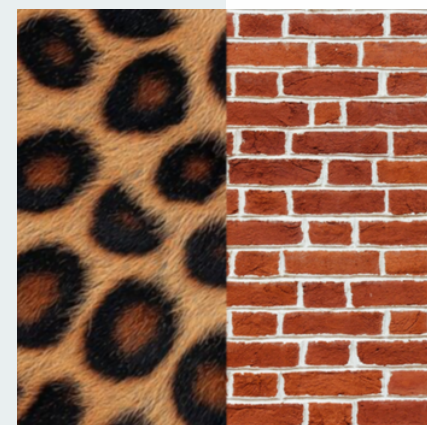
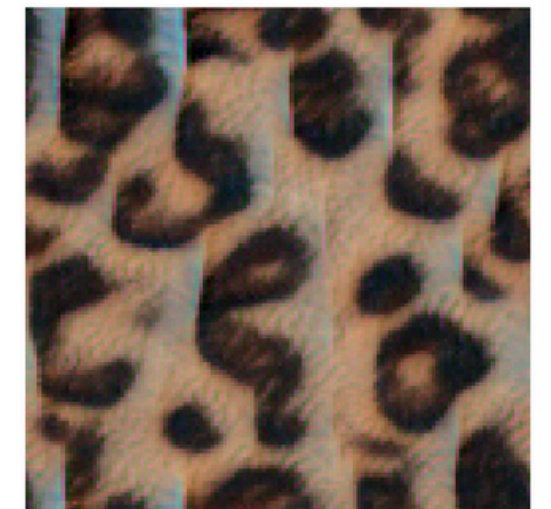
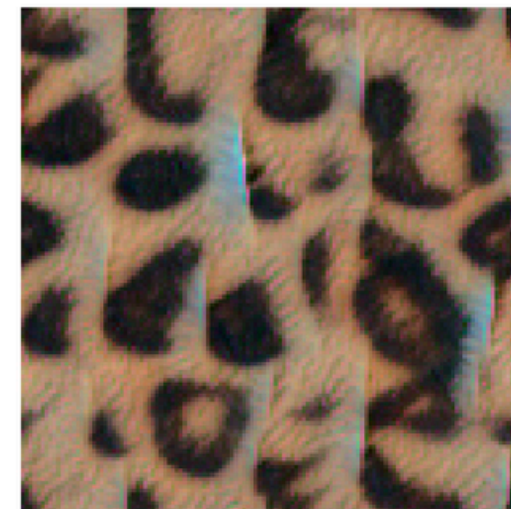
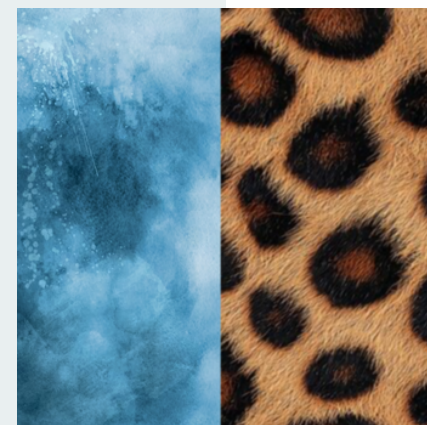


FIGURE 14 – Reconstruction de la texture bubbles après dommage circulaire.

Tests et conclusions

Nombre de textures par image



Limitations

Images globalement structurées

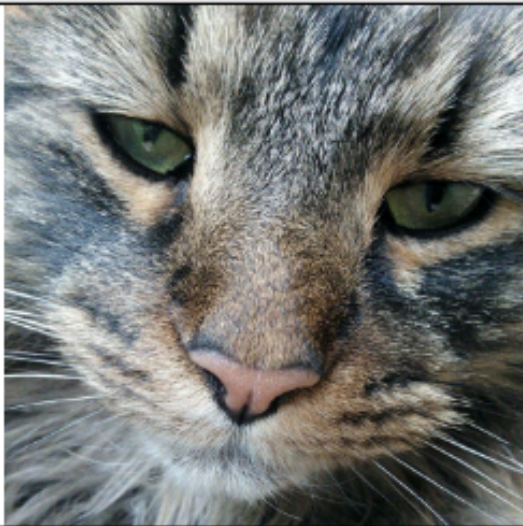
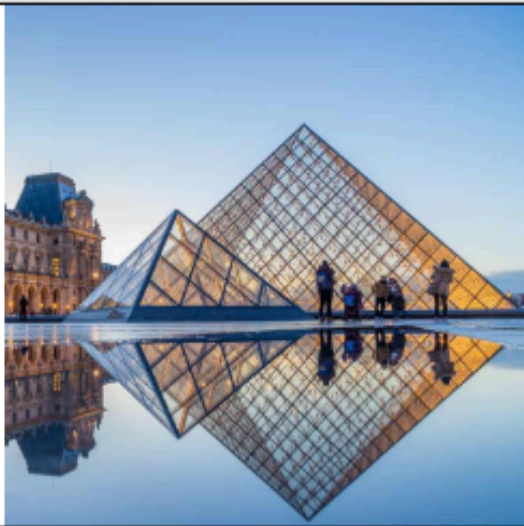
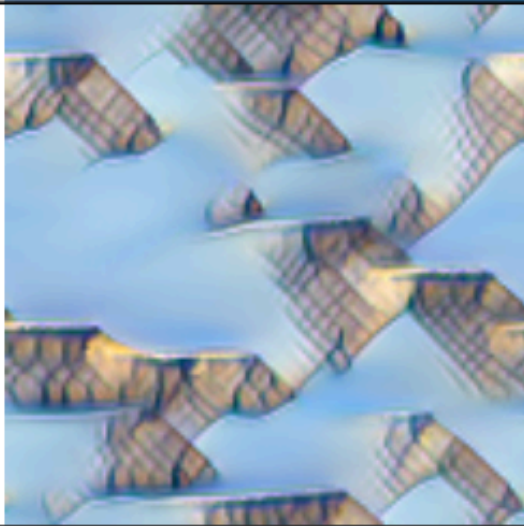
	chat	louvre	biscuits
Image d'origine			
10 000 steps			

TABLE 4 – Tests sur images **globalement** structurées avec l'architecture mono-texture finale.

Limitations

Structures et couleurs

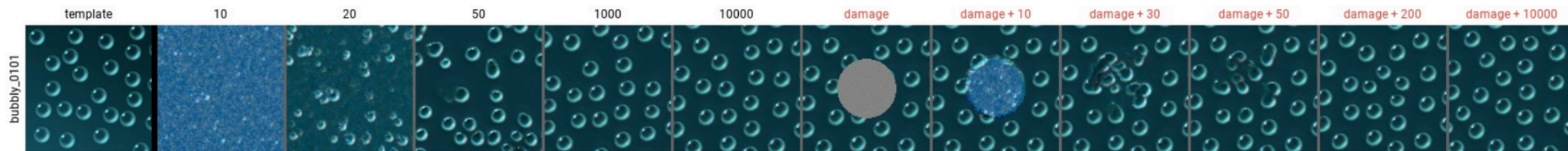


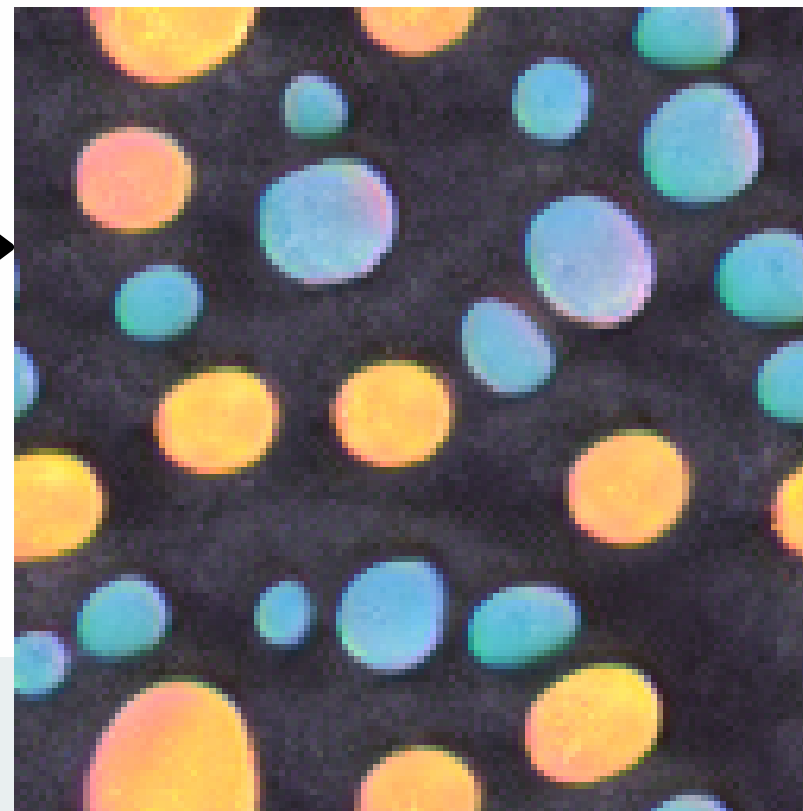
FIGURE 15 – Résultat obtenu par Mordvintsev et. al [3] sur la texture "bubbles".

Difficultés à reproduire une grande **variété de couleurs** et de tailles d'objet.

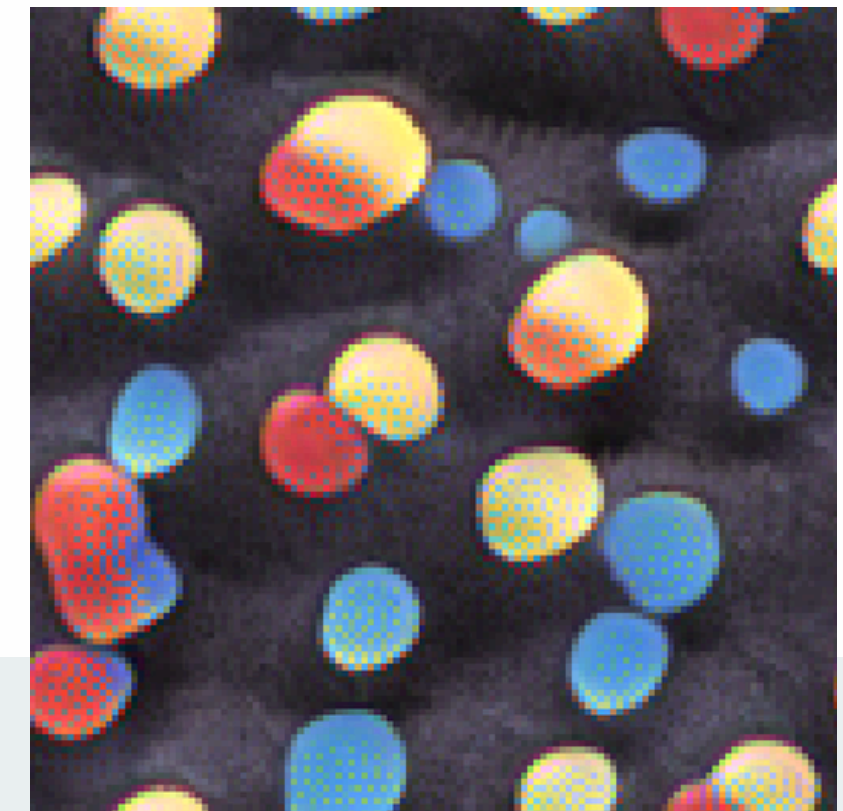
Harmonisation de la texture source.



Gram

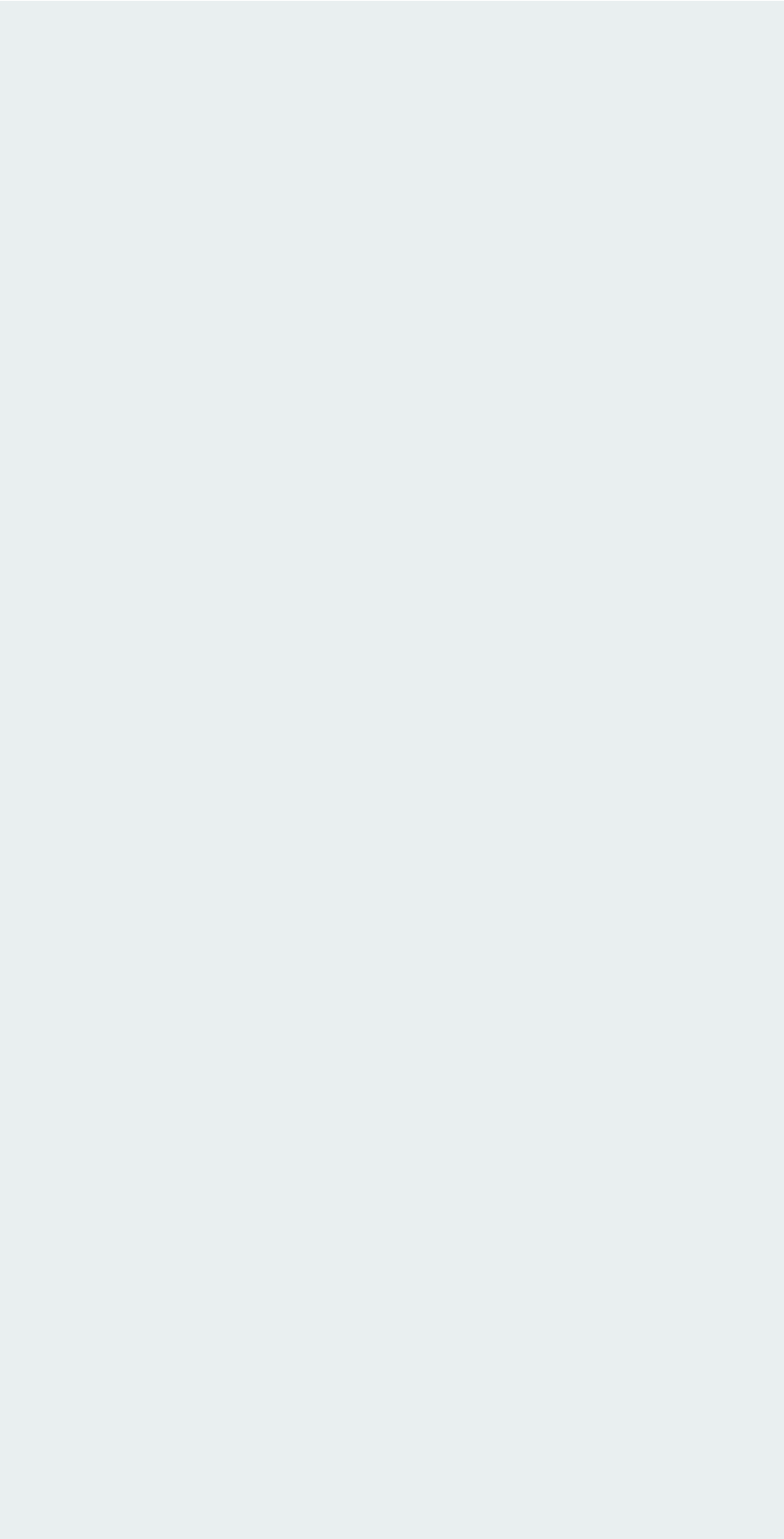


SOT



CONCLUSION





ANNEXE

Limitations

Taille des filtres

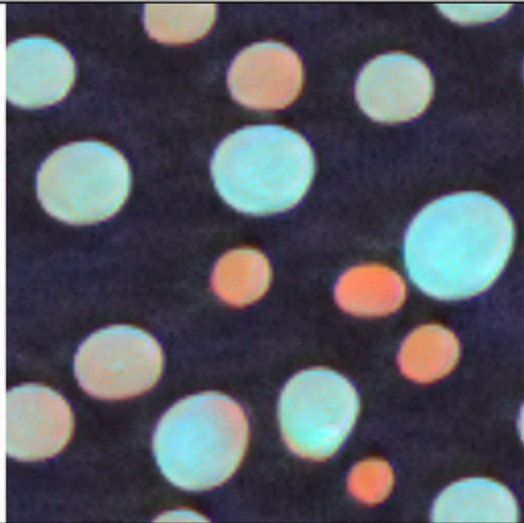
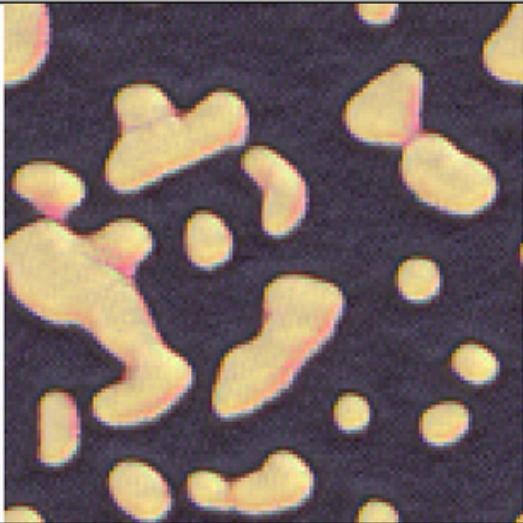
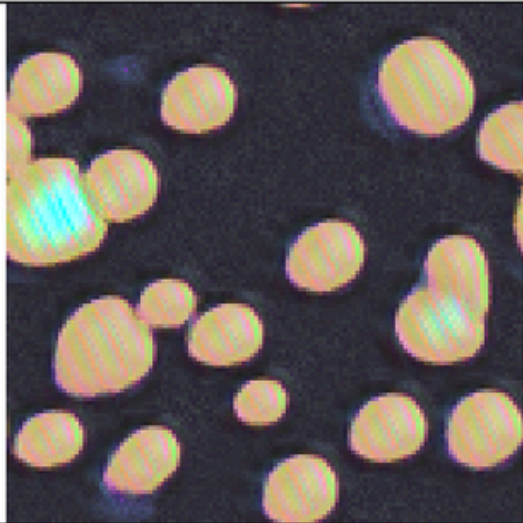
	Filtres 3×3 de l'article	Filtres aléatoires 3×3	Filtres aléatoires 7×7
dotted			
biscuits			

TABLE 5 – Tests de filtres de plus grande taille.